

PHASE 2 — GIT & VERSION CONTROL COMMAND REFERENCE

git config

Purpose: Configures Git user information and other Git settings.

Example:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

git config --list

Purpose: Displays the current Git configuration.

Example:

```
git config --list
```

git --version

Purpose: Displays the installed Git version.

Example:

```
git --version
```

git init

Purpose: Creates a new Git repository in the current directory.

Example:

```
git init
```

git clone

Purpose: Creates a local copy of an existing remote Git repository.

Example:

```
git clone https://github.com/username/repository.git
```

git status

Purpose: Displays the current state of the working directory and staging area.

Example:

```
git status
```

git add

Purpose: Adds changes to the staging area before committing them.

Example:

```
git add file.txt
```

Add all changed files:

```
git add .
```

git commit

Purpose: Records staged changes in the Git repository.

Example:

```
git commit -m "Add Linux command reference"
```

git commit --amend

Purpose: Modifies the most recent commit.

Example:

```
git commit --amend -m "Updated commit message"
```

git log

Purpose: Displays the commit history of a repository.

Example:

```
git log
```

git log --oneline

Purpose: Displays a compact version of the commit history.

Example:

```
git log --oneline
```

git show

Purpose: Displays detailed information about a specific commit.

Example:

```
git show COMMIT_ID
```

git diff

Purpose: Displays changes that have not yet been staged.

Example:

```
git diff
```

git diff --staged

Purpose: Displays changes that have been added to the staging area.

Example:

```
git diff --staged
```

git branch

Purpose: Lists, creates, or manages Git branches.

Example:

```
git branch
```

Create a branch:

```
git branch feature-login
```

git switch

Purpose: Switches between Git branches.

Example:

```
git switch feature-login
```

git switch -c

Purpose: Creates a new branch and switches to it immediately.

Example:

```
git switch -c feature-login
```

git checkout

Purpose: Switches branches or restores files.

Example:

```
git checkout main
```

Create and switch to a branch:

```
git checkout -b feature-login
```

git merge

Purpose: Combines changes from one branch into another branch.

Example:

```
git switch main  
git merge feature-login
```

git rebase

Purpose: Reapplies commits from one branch on top of another branch to create a linear project history.

Example:

```
git switch feature-login  
git rebase main
```

git fetch

Purpose: Downloads new commits and references from a remote repository without modifying the current working branch.

Example:

```
git fetch origin
```

git pull

Purpose: Downloads changes from a remote repository and integrates them into the current branch.

Example:

```
git pull origin main
```

git push

Purpose: Uploads local commits to a remote repository.

Example:

```
git push origin main
```

git push -u

Purpose: Pushes a local branch to a remote repository and sets its upstream branch.

Example:

```
git push -u origin main
```

git remote

Purpose: Manages connections to remote Git repositories.

Example:

```
git remote -v
```

git remote add

Purpose: Adds a remote repository to the local Git repository.

Example:

```
git remote add origin https://github.com/username/repository.git
```

git remote remove

Purpose: Removes a configured remote repository.

Example:

```
git remote remove origin
```

git remote rename

Purpose: Renames an existing remote repository reference.

Example:

```
git remote rename origin upstream
```

git tag

Purpose: Creates, lists, or manages tags used to mark specific points in Git history.

Example:

```
git tag
```

Create a tag:

```
git tag v1.0.0
```

git tag -a

Purpose: Creates an annotated tag containing additional metadata and a message.

Example:

```
git tag -a v1.0.0 -m "Version 1.0.0"
```

git push --tags

Purpose: Uploads local Git tags to the remote repository.

Example:

```
git push origin --tags
```

Undo and Recovery Commands

git restore

Purpose: Restores files to a previous state and can remove unstaged changes.

Example:

```
git restore file.txt
```

git restore --staged

Purpose: Removes a file from the staging area without deleting its changes.

Example:

```
git restore --staged file.txt
```

git reset

Purpose: Moves the current branch pointer and can modify the staging area or working tree depending on the option used.

Example:

```
git reset HEAD~1
```

git reset --soft

Purpose: Moves the branch pointer while keeping changes staged.

Example:

```
git reset --soft HEAD~1
```

git reset --mixed

Purpose: Moves the branch pointer and unstages changes while keeping the working files unchanged.

Example:

```
git reset --mixed HEAD~1
```

git reset --hard

Purpose: Resets the repository, staging area, and working directory to a specified commit.

Example:

```
git reset --hard HEAD~1
```

Use carefully because uncommitted changes can be lost.

git revert

Purpose: Creates a new commit that reverses the changes introduced by an earlier commit.

Example:

```
git revert COMMIT_ID
```

git reflog

Purpose: Displays a history of changes to local branch references and helps recover commits or states that are no longer visible in the normal history.

Example:

```
git reflog
```

Branch Management

git branch -a

Purpose: Displays local and remote branches.

Example:

```
git branch -a
```

git branch -d

Purpose: Deletes a local branch that has been merged.

Example:

```
git branch -d feature-login
```

git branch -D

Purpose: Force-deletes a local branch.

Example:

```
git branch -D feature-login
```

git branch -m

Purpose: Renames the current branch.

Example:

```
git branch -m old-name new-name
```

git merge --abort

Purpose: Cancels an ongoing merge when conflicts cannot or should not be resolved.

Example:

```
git merge --abort
```

git rebase --abort

Purpose: Cancels an ongoing rebase operation.

Example:

```
git rebase --abort
```

git rebase --continue

Purpose: Continues a rebase after resolving a conflict.

Example:

```
git rebase --continue
```

Merge Conflicts

git status

Purpose: Shows files that are involved in a merge conflict.

Example:

```
git status
```

git add

Purpose: Marks a conflicted file as resolved after manually fixing the conflict.

Example:

```
git add file.txt
```

git commit

Purpose: Creates a commit after resolving merge conflicts during a merge.

Example:

```
git commit -m "Resolve merge conflict"
```

Stash

git stash

Purpose: Temporarily saves uncommitted changes without committing them.

Example:

```
git stash
```

git stash list

Purpose: Displays all saved Git stashes.

Example:

```
git stash list
```

git stash pop

Purpose: Restores the most recent stashed changes and removes that stash from the stash list.

Example:

```
git stash pop
```

git stash apply

Purpose: Restores stashed changes while keeping the stash in the stash list.

Example:

```
git stash apply
```

git stash drop

Purpose: Deletes a specific stash.

Example:

```
git stash drop
```

Git Ignore

.gitignore

Purpose: Specifies files and directories that Git should not track.

Example (.gitignore):

```
node_modules/  
.env  
*.log  
__pycache__/  
*.tmp
```

git check-ignore

Purpose: Checks whether a file is being ignored by Git.

Example:

```
git check-ignore -v .env
```

Git Information

git status -sb

Purpose: Displays a short summary of the current branch and working-tree status.

Example:

```
git status -sb
```

git log --graph

Purpose: Displays the commit history as a graphical branch structure.

Example:

```
git log --oneline --graph --all
```

git shortlog

Purpose: Summarizes commits by author.

Example:

```
git shortlog
```

git describe

Purpose: Produces a human-readable name for a commit based on the nearest tag.

Example:

```
git describe
```

GitHub Workflow

git clone

Purpose: Downloads an existing GitHub repository to the local machine.

Example:

```
git clone https://github.com/username/project.git
```

git switch -c

Purpose: Creates a new feature branch for development.

Example:

```
git switch -c feature/security-update
```

git add

Purpose: Stages completed changes.

Example:

```
git add .
```

git commit

Purpose: Creates a snapshot of the staged changes with a descriptive message.

Example:

```
git commit -m "Add security documentation"
```

git push

Purpose: Uploads the feature branch to GitHub.

Example:

```
git push -u origin feature/security-update
```

Conventional Commits

Conventional Commit Format

Purpose: Provides a standardized format for writing clear and consistent commit messages.

Format:

```
type: description
```

Examples:

```
feat: add health check script
fix: resolve nginx configuration issue
docs: update Linux command reference
refactor: simplify backup script
test: add service health tests
```


chore: update dependencies

Semantic Versioning

Semantic Versioning

Purpose: Provides a standard way to represent software versions.

Format:

MAJOR.MINOR.PATCH

Example:

1.0.0

MAJOR: Introduces incompatible changes.

MINOR: Adds backward-compatible functionality.

PATCH: Introduces backward-compatible bug fixes.

Git Tags

git tag

Purpose: Lists existing tags in a repository.

Example:

git tag

git tag v1.0.0

Purpose: Creates a lightweight tag for a specific point in the Git history.

Example:

git tag v1.0.0

git tag -a

Purpose: Creates an annotated release tag with a message.

Example:

git tag -a v1.0.0 -m "First stable release"

Git Security

git grep

Purpose: Searches tracked files in a Git repository for specific text.

Example:

git grep "password"

git log -p

Purpose: Displays commit history together with the changes introduced by each commit.

Example:

git log -p

git log --all

Purpose: Displays commit history from all local branches.

Example:

```
git log --all --oneline
```

git clean

Purpose: Removes untracked files from the working directory.

Example:

```
git clean -n
```

The -n option performs a dry run without deleting anything.

git clean -fd

Purpose: Removes untracked files and directories.

Example:

```
git clean -fd
```

Use carefully because these files are not tracked by Git and may be permanently removed.

Git Credential and Authentication Concepts

SSH Authentication

Purpose: Allows GitHub authentication using an SSH key instead of entering a username and password for every operation.

Example:

```
git remote set-url origin git@github.com:username/repository.git
```

git remote -v

Purpose: Displays the URLs associated with the repository's remote connections.

Example:

```
git remote -v
```

GitHub Security Concepts

Secret Scanning

Purpose: Detects potentially exposed credentials, API keys, tokens, and other sensitive information committed to repositories.

Important concept: Deleting a secret from the latest version of a file does not necessarily remove it from Git history.

If a real secret is exposed, it should be revoked or rotated immediately, and the secret should also be removed from repository history when appropriate.

Token Hygiene

Purpose: Ensures that authentication tokens are stored securely and are not committed to Git repositories.

Good practice — never commit:

- .env files
- API keys
- Passwords
- Private keys
- Access tokens
- Cloud credentials

Important Git Concepts

Working Directory

Purpose: The files and folders currently being edited on your local machine.

Staging Area

Purpose: A temporary area where changes are selected before they are committed.

Command:

```
git add .
```

Local Repository

Purpose: The Git repository stored on your local computer containing commits and history.

Remote Repository

Purpose: A Git repository hosted on a remote platform such as GitHub.

Commit

Purpose: A permanent snapshot of staged changes in the repository's history.

Branch

Purpose: Provides an independent line of development within a Git repository.

Merge

Purpose: Combines changes from different branches.

Rebase

Purpose: Reapplies commits onto another base commit to create a cleaner, more linear history.

Pull Request

Purpose: A GitHub mechanism for proposing changes from one branch to another and enabling review before merging.

Protected Branch

Purpose: Prevents direct or unauthorized changes to important branches such as main and can require reviews or status checks before merging.

Standard Git Workflow

Step 1 — Clone Repository

Purpose: Creates a local copy of the remote repository.

Command:

```
git clone REPOSITORY_URL
```

Step 2 — Create Feature Branch

Purpose: Creates an isolated branch for a new task or feature.

Command:

```
git switch -c feature-name
```

Step 3 — Check Status

Purpose: Checks modified and untracked files.

Command:

```
git status
```

Step 4 — Stage Changes

Purpose: Adds selected changes to the staging area.

Command:

```
git add .
```

Step 5 — Commit Changes

Purpose: Records the staged changes.

Command:

```
git commit -m "feat: add new feature"
```

Step 6 — Push Branch

Purpose: Uploads the feature branch to the remote repository.

Command:

```
git push -u origin feature-name
```

Step 7 — Create Pull Request

Purpose: Requests review of the feature branch before merging it into the main branch.

Step 8 — Merge

Purpose: Integrates the approved changes into the main branch.

Step 9 — Pull Latest Changes

Purpose: Updates the local repository with the latest changes from the remote repository.

Command:

```
git pull origin main
```